

Morphological Analysis & Language Modeling MODULE-II

Table of Contents

- Introduction to Morphology
- Importance of Morphological Analysis
- Key Techniques used in Morphological Analysis
- Types of Morphology
- Applications of Morphology in NLP
- A regular expression
- Language Model
- Types of LM
- Morphological Analysis in NLP

Introduction to Morphology

- Morphology is the study of word formation and structure.
- It helps in understanding how words are formed, modified, and related.
- In NLP, morphology plays a crucial role in text processing and language understanding.

Importance of Morphological Analysis

- **Understanding Word Formation:** It helps in identifying the basic building blocks of words, which is crucial for language comprehension.
- **Improving Text Analysis:** By breaking down words into their roots and affixes, it enhances the accuracy of text analysis tasks like sentiment analysis and topic modeling.
- **Enhancing Language Models:** Morphological analysis provides detailed insights into word formation, improving the performance of language models used in tasks like speech recognition and text generation.
- **Facilitating Multilingual Processing:** It aids in handling the morphological diversity of different languages, making NLP systems more robust and versatile.

Key Techniques used in Morphological Analysis

1. Stemming

- [Stemming](#) reduces words to their base or root form, usually by removing suffixes. The resulting stems are not necessarily valid words but are useful for text normalization.

2. Lemmatization

- [Lemmatization](#) reduces words to their base or dictionary form (lemma). It considers the context and part of speech, producing valid words.

Key Techniques used in Morphological Analysis

3. Morphological Parsing

- Morphological parsing involves analyzing the structure of words to identify their morphemes (roots, prefixes, suffixes). It requires knowledge of morphological rules and patterns. Finite-State Transducers (FSTs) is uses as a tool for morphological parsing.

4. Neural Network Models

- Neural network models, especially deep learning models, can be trained to perform morphological analysis by learning patterns from large datasets.

Key Techniques used in Morphological Analysis

- **Types of Neural Network**
- **Recurrent Neural Networks (RNNs)**: Useful for sequential data like text.
- **Convolutional Neural Networks (CNNs)**: Can capture local patterns in the text.
- **Transformers**: Advanced models like BERT and GPT that understand context and semantics.

Key Techniques used in Morphological Analysis

5. Rule-Based Methods

- Rule-based methods rely on manually defined linguistic rules for morphological analysis. These rules can handle specific language patterns and exceptions.

6. Hidden Markov Models (HMMs)

- Hidden Markov Models (HMMs) are probabilistic models that can be used to analyze sequences of data, such as morphemes in words. HMMs consist of a set of hidden states, each representing a possible state of the system, and observable outputs generated from these states.

Types of Morphology

- Inflectional Morphology: Changes a word's form without altering its core meaning.
- Derivational Morphology: Creates new words by adding prefixes or suffixes, changing the meaning or grammatical category.

Inflectional Morphology

- Inflectional morphology conveys grammatical information, such as number, tense, agreement or case.
- Due to inflectional morphological process, the meaning and categories of the new inflected words usually do not change. That is, a noun can be inflected to a noun while adding affixes, a verb can be inflected to a verb in different tense in English.
- We would say, the root word (stem) is **inflected** to form other words of same meaning and category.
- Inflection creates different forms of the same word.
- In English, only nouns and verbs can be inflected (sometimes adjectives too). Inflectional morphemes are very less when compared with some other languages.

Inflectional Morphology

- Involves modifications like tense, number, gender, case, etc.
- Does not change the word's fundamental meaning or part of speech.
- Examples:
 - - 'talk' → 'talked' (past tense)
 - - 'cat' → 'cats' (plural form)

Derivational Morphology

- Creates new words by adding prefixes or suffixes.
- Can change the grammatical category of the word.
- Examples:
 - - 'happy' → 'happiness' (adjective to noun)
 - - 'act' → 'react' (changes meaning)

Derivational Morphology

- It defined as morphology that creates new lexemes, either by changing the syntactic category (part of speech) of a base or by adding substantial, nongrammatical meaning or both.
- derivation may be distinguished from inflectional morphology, which typically does not change category but rather modifies lexemes to fit into various syntactic contexts; inflection typically expresses distinctions like number, case, tense, aspect, person, among others.
- derivation may be distinguished from compounding, which also creates new lexemes, but by combining two or more bases rather than by affixation, reduplication, subtraction, or internal modification of various sorts.

Applications of Morphology in NLP

- **Information Retrieval:** Enhances search engines by improving the matching of query terms with relevant documents, even if they are in different morphological forms.
- **Machine Translation:** Facilitates accurate translation by understanding and generating correct word forms in different languages.
- **Text-to-Speech Systems:** Improves pronunciation and intonation by accurately identifying word structures and their stress patterns.
- **Spell Checkers and Grammar Checkers:** Detects and suggests corrections for misspelled words and grammatical errors by analyzing word forms and their usage.
- **Named Entity Recognition (NER):** Helps in identifying and classifying named entities in text by understanding their morphological variations.

A regular expression

- A regular expression (sometimes called a rational expression) is a sequence of characters that define a search pattern, mainly for use in pattern matching with strings, or string matching, i.e. “find and replace” like operations. Regular expressions are a generalized way to match patterns with sequences of characters

How to write regular expressions?

- There are certain elements used to write regular expressions as mentioned below:
- **1. Repeaters (*, +, and { })**
- These symbols act as repeaters and tell the computer that the preceding character is to be used for more than just one time.
- **2. The asterisk symbol (*)**
- It tells the computer to match the preceding character (or set of characters) for 0 or more times (upto infinite).
- Example The regular expression `ab*c` will give `ac`, `abc`, `abbc`, `abbbc`....and so on

How to write regular expressions?

3. The Plus symbol (+)

- It tells the computer to repeat the preceding character (or set of characters) at least one or more times (up to infinite).
- **Example :** The regular expression `ab+c` will give `abc`, `abbc`, `abbbc`, ... and so on.

4. The curly braces { ... }

- It tells the computer to repeat the preceding character (or set of characters) for as many times as the value inside this bracket.
- **Example :** `{2}` means that the preceding character is to be repeated 2 times, `{min,}` means the preceding character matches min or more times. `{min,max}` means that the preceding character is repeated at least min & at most max times.

5. Wildcard (.)

- The dot symbol can take the place of any other symbol, that is why it is called the wildcard character.
- **Example :**
The Regular expression `.*` will tell the computer that any character can be used any number of times.

How to write regular expressions?

6.Optional character (?)

- This symbol tells the computer that the preceding character may or may not be present in the string to be matched.
- **Example :**
We may write the format for document file as – “docx?”
The ‘?’ tells the computer that x may or may not be present in the name of file format.

7. The caret (^) symbol (*Setting position for the match*)

- The caret symbol tells the computer that the match must start at the beginning of the string or line.
- **Example :** `^\d{3}` will match with patterns like "901" in "901-333-".

How to write regular expressions?

8. The dollar (\$) symbol

- It tells the computer that the match must occur at the end of the string or before `\n` at the end of the line or string.
- **Example** : `-\d{3}$` will match with patterns like `"-333"` in `"-901-333"`.

9.Character Classes

- A character class matches any one of a set of characters. It is used to match the most basic element of a language like a letter, a digit, a space, a symbol, etc.
- **Example** : `[abc]` will match characters a,b and c in any string.

How to write regular expressions?

10. `[^set_of_characters]` *Negation:*

- Matches any single character that is not in `set_of_characters`. By default, the match is case-sensitive.
- **Example :** `[^abc]` will match any character except a,b,c .

11. `[first-last]` *Character range:*

- Matches any single character in the range from first to last.
- **Example :** `[a-zA-Z]` will match any character from a to z or A to Z.

12. The Escape Symbol (`\`)

- If you want to match for the actual '+', '.' etc characters, add a backslash(`\`) before that character. This will tell the computer to treat the following character as a search character and consider it for a matching pattern.
- **Example :** `\d+[\+-x\*]\d+` will match patterns like "2+2" and "3*9" in "(2+2) * 3*9".

How to write regular expressions?

13. Grouping Characters ()

- A set of different symbols of a regular expression can be grouped together to act as a single unit and behave as a block, for this, you need to wrap the regular expression in the parenthesis().
- **Example :** ([A-Z]\w+) contains two different elements of the regular expression combined together. This expression will match any pattern containing uppercase letter followed by any character.

14. Vertical Bar (|)

- Matches any one element separated by the vertical bar (|) character.
- **Example :** th(e|is|at) will match words - the, this and that.

15. \number

- *Backreference*: allows a previously matched sub-expression(expression captured or enclosed within circular brackets) to be identified subsequently in the same regular expression. \n means that group enclosed within the n-th bracket will be repeated at current position.
- **Example :** ([a-z])\1 will match “ee” in Geek because the character at second position is same as character at position 1 of the match

Language model

What is a language model?

A language model is a machine learning [model](#) that aims to predict and generate plausible language. Autocomplete is a language model, for example.

These models work by estimating the probability of a [token](#) or sequence of tokens occurring within a longer sequence of tokens. Consider the following sentence:

EXAMPLE

When I hear rain on my roof, I _____ in my kitchen.

If you assume that a token is a word, then a language model determines the probabilities of different words or sequences of words to replace that underscore.

For example, a language model might determine the following probabilities:

cook soup 9.4% warm up a kettle 5.2% cower 3.6% nap 2.5% relax 2.2%

Language model

- Language models are a fundamental component of natural language processing (NLP) and computational linguistics. They are designed to understand, generate, and predict human language. These models analyze the structure and use of language to perform tasks such as machine translation, text generation, and sentiment analysis.

What is a Language Model in Natural Language Processing?

- A language model in natural language processing (NLP) is a statistical or machine learning model that is used to predict the next word in a sequence given the previous words. Language models play a crucial role in various NLP tasks such as machine translation, speech recognition, text generation, and sentiment analysis. They analyze and understand the structure and use of human language, enabling machines to process and generate text that is contextually appropriate and coherent

Types of LM

- **Language models can be broadly categorized into two types:**
 - Pure Statistical Methods
 - Neural Models
- **Purpose and Functionality**
- The primary purpose of a language model is to capture the statistical properties of natural language. By learning the probability distribution of word sequences, a language model can predict the likelihood of a given word following a sequence of words. This predictive capability is fundamental for tasks that require understanding the context and meaning of text.
- For instance, in text generation, a language model can generate plausible and contextually relevant text by predicting the next word in a sequence iteratively. In machine translation, language models help in translating text from one language to another by understanding and generating grammatically correct sentences in the target language.

Pure Statistical Methods

- Pure statistical methods form the basis of traditional language models. These methods rely on the statistical properties of language to predict the next word in a sentence, given the previous words.

They include

- n-grams,
- exponential models, and
- skip-gram models.

1. N-grams

- An n-gram is a sequence of n items from a sample of text or speech, such as phonemes, syllables, letters, words, or base pairs. N-gram models use the frequency of these sequences in a training corpus to predict the likelihood of word sequences.
- For example, a bigram (2-gram) model predicts the next word based on the previous word, while a trigram (3-gram) model uses the two preceding words.
- N-gram models are simple, easy to implement, and computationally efficient, making them suitable for applications with limited computational resources.
- they have significant limitations. They struggle with capturing long-range dependencies due to their limited context window.
- As n increases, the number of possible n-grams grows exponentially, leading to sparsity issues where many sequences are never observed in the training data. This sparsity makes it difficult to accurately estimate the probabilities of less common sequences.

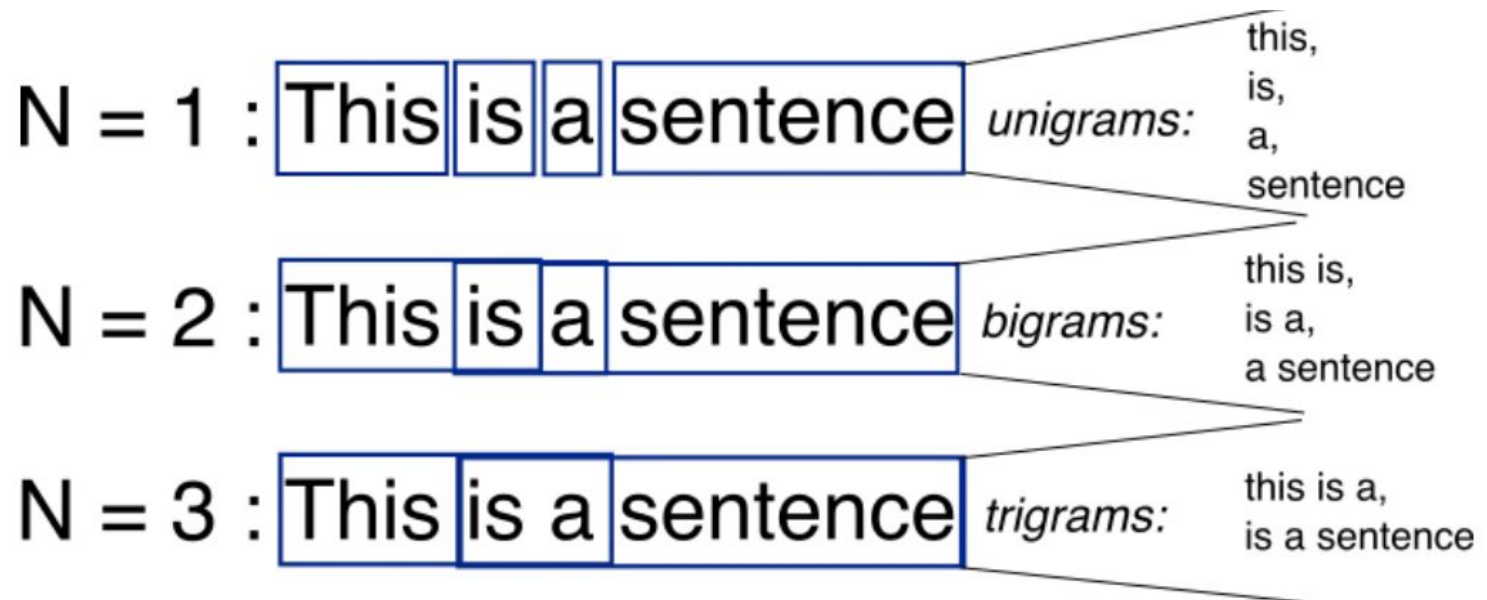
N-grams

- N-grams are contiguous sequences of 'n' items, typically words in the context of NLP. These items can be characters, words, or even syllables, depending on the granularity desired. The value of 'n' determines the order of the N-gram.

Examples:

- **Unigrams (1-grams):** Single words, e.g., “cat,” “dog.”
- **Bigrams (2-grams):** Pairs of consecutive words, e.g., “natural language,” “deep learning.”
- **Trigrams (3-grams):** Triplets of consecutive words, e.g., “machine learning model,” “data science approach.”
- 4-grams, 5-grams, etc.: Sequences of four, five, or more consecutive words.

N-grams



Significance of N-grams in NLP

1. Capturing Context and Semantics:

- N-grams help capture the contextual information and semantics within a sequence of words, providing a more nuanced understanding of language.

2. Improving Language Models:

- In language modeling tasks, N-grams contribute to building more accurate and context-aware models, enhancing the performance of applications such as machine translation and speech recognition.

3. Enhancing Text Prediction:

- N-grams are essential for predictive text applications, aiding in the prediction of the next word or sequence of words based on the context provided by the preceding N-gram.

4. Information Retrieval:

- In information retrieval tasks, N-grams assist in matching and ranking documents based on the relevance of N-gram patterns.

5. Feature Extraction:

- N-grams serve as powerful features in text classification and sentiment analysis, capturing meaningful patterns that contribute to the characterization of different classes or sentiments.

2. Exponential Models

- Exponential models, such as the Maximum Entropy model, are more flexible and powerful than n-gram models. They predict the probability of a word based on a wide range of features, including not only the previous words but also other contextual information. These models assign weights to different features and combine them using an exponential function to estimate probabilities.
- **Maximum Entropy Models**
- Maximum Entropy (MaxEnt) models, also known as logistic regression in the context of classification, are used to estimate the probabilities of different outcomes based on a set of features. In the context of language modeling, MaxEnt models use features such as the presence of certain words, part-of-speech tags, and syntactic patterns to predict the next word. The model parameters are learned by maximizing the likelihood of the observed data under the model.
- MaxEnt models are more flexible than n-gram models because they can incorporate a wider range of features. However, they are also more complex and computationally intensive to train. Like n-gram models, MaxEnt models still struggle with long-range dependencies because they rely on fixed-length context windows.

3.Skip-gram Model

- Skip-gram models are a type of statistical method used primarily in word embedding techniques.
- They predict the context words (surrounding words) given a target word within a certain window size. Skip-gram models, particularly those used in Word2Vec, are effective for capturing the semantic relationships between words by optimizing the likelihood of context words appearing around a target word.
- **Word2Vec and Skip-gram**
- Word2Vec, developed by Google, includes two main architectures: skip-gram and continuous bag-of-words (CBOW).
- The skip-gram model predicts the context words given a target word, while the CBOW model predicts the target word given the context words. Both models are trained using neural networks, but they are conceptually simple and computationally efficient.

Neural Models

- Neural models have revolutionized the field of NLP by leveraging deep learning techniques to create more sophisticated and accurate language models. These models include
 - Recurrent Neural Networks (RNNs),
 - Transformer-based models, and
 - large language models.

1.Recurrent Neural Networks

- Recurrent Neural Networks (RNNs) are a type of neural network designed for sequential data, making them well-suited for language modelling.
- RNNs maintain a hidden state that captures information about previous inputs, allowing them to consider the context of words in a sequence.
- LSTMs and GRUs are advanced RNN variants that address the vanishing gradient problem, enabling the capture of long-range dependencies in text.
- LSTMs use a gating mechanism to control the flow of information, while GRUs simplify the gating mechanism, making them faster to train.

2. Transformer-based Models

- The Transformer model, introduced by Vaswani et al. in 2017, has revolutionized NLP. Unlike RNNs, which process data sequentially, the Transformer model processes the entire input simultaneously, making it more efficient for parallel computation.
- The key components of the Transformer architecture are:
- **Self-Attention Mechanism:** This mechanism allows the model to weigh the importance of different words in a sequence, capturing dependencies regardless of their distance in the text. Each word's representation is updated based on its relationship with all other words in the sequence.
- **Encoder-Decoder Structure:** The Transformer consists of an encoder and a decoder. The encoder processes the input sequence and generates a set of hidden representations. The decoder takes these representations and generates the output sequence.
- **Positional Encoding:** Since Transformers do not process the input sequentially, they use positional encoding to retain information about the order of words in a sequence. This encoding adds positional information to the input embeddings, allowing the model to consider the order of words.

3. Large language models (LLMs)

- Large language models have pushed the boundaries of what is possible in NLP. These models are characterized by their vast size, often comprising billions of parameters, and their ability to perform a wide range of tasks with minimal fine-tuning.
- Training large language models involves feeding them vast amounts of text data and optimizing their parameters using powerful computational resources. The training process typically includes multiple stages, such as unsupervised pre-training on large corpora followed by supervised fine-tuning on specific tasks.
- While large language models offer remarkable performance, they also pose significant challenges. Training these models requires substantial computational resources and energy, raising concerns about their environmental impact. Additionally, the models' size and complexity can make them difficult to interpret and control, leading to potential ethical and bias issues.

Popular Language Models in NLP

- Several language models have gained prominence due to their innovative architecture and impressive performance on NLP tasks.
- Here are some of the most notable models:
- **BERT (Bidirectional Encoder Representations from Transformers)**
 - BERT, developed by Google, is a Transformer-based model that uses bidirectional context to understand the meaning of words in a sentence. It has improved the relevance of search results and achieved state-of-the-art performance in many NLP benchmarks.

Popular Language Models in NLP

GPT-3 (Generative Pre-trained Transformer 3)

- GPT-3, developed by OpenAI, is a large language model known for its ability to generate coherent and contextually appropriate text based on a given prompt. With 175 billion parameters, it is one of the largest and most powerful language models to date.

T5 (Text-to-Text Transfer Transformer)

- T5, developed by Google, treats all NLP tasks as a text-to-text problem, enabling it to handle a wide range of tasks with a single model. It has demonstrated versatility and effectiveness across various NLP tasks.

Word2Vec

- Word2Vec, developed by Google, includes the skip-gram and continuous bag-of-words (CBOW) models. These models create word embeddings that capture semantic similarities between words, improving the performance of downstream NLP tasks.

Popular Language Models in NLP

ELMo (Embeddings from Language Models)

- ELMo generates context-sensitive word embeddings by considering the entire sentence. It uses bidirectional LSTMs and has improved performance on various NLP tasks by providing more nuanced word representations.

Transformer-XL

- Transformer-XL is an extension of the Transformer model that addresses the fixed-length context limitation by introducing a segment-level recurrence mechanism. This allows the model to capture longer-range dependencies more effectively.

XLNet

- XLNet, developed by Google, is an autoregressive Transformer model that uses permutation-based training to capture bidirectional context. It has achieved state-of-the-art results on several NLP benchmarks.

Popular Language Models in NLP

RoBERTa (Robustly Optimized BERT Approach)

- RoBERTa, developed by Facebook AI, is a variant of BERT that uses more extensive training data and optimizations to achieve better performance. It has set new benchmarks in several NLP tasks.

ALBERT (A Lite BERT)

- ALBERT, developed by Google, is a lightweight version of BERT that reduces the model size while maintaining performance. It achieves this by sharing parameters across layers and factorizing the embedding parameters.

Turing-NLG

- Turing-NLG, developed by Microsoft, is a large language model known for its ability to generate high-quality text. It has been used in various applications, including chatbots and virtual assistants.

Implementing N-Gram Language Modelling in NLTK

- **N-gram**
- N-gram can be defined as the contiguous sequence of n items from a given sample of text or speech. The items can be letters, words, or base pairs according to the application. The N-grams typically are collected from a text or speech corpus (A long text dataset).
- For instance, N-grams can be unigrams like ("This", "article", "is", "on", "NLP") or bigrams ("This article", "article is", "is on", "on NLP").
- **N-gram Language Model**
- An N-gram language model predicts the probability of a given N-gram within any sequence of words in a language. A well-crafted N-gram model can effectively predict the next word in a sentence, which is essentially determining the value of $p(w | h)$, where h is the history or context and w is the word to predict.

Implementing N-Gram Language Modelling in NLTK

```
# Import necessary libraries
import nltk
from nltk import bigrams, trigrams
from nltk.corpus import reuters
from collections import defaultdict

# Download necessary NLTK resources
nltk.download('reuters')
nltk.download('punkt')

# Tokenize the text
words = nltk.word_tokenize(' '.join(reuters.words()))

# Create trigrams
tri_grams = list(trigrams(words))

# Build a trigram model
model = defaultdict(lambda: defaultdict(lambda: 0))

# Count frequency of co-occurrence
for w1, w2, w3 in tri_grams:
    model[(w1, w2)][w3] += 1
```

```
# To find the probability of a trigram
```

Implementing N-Gram Language Modelling in NLTK

Function to predict the next word

```
def predict_next_word(w1, w2):
```

```
    """
```

Predicts the next word based on the previous two words using the trained trigram model.

Args:

w1 (str): The first word.

w2 (str): The second word.

Returns:

str: The predicted next word.

```
    """
```

```
    next_word = model[w1, w2]
```

```
    if next_word:
```

```
        predicted_word = max(next_word, key=next_word.get) # Choose the most likely next word
```

```
    return predicted_word else:
```

```
        return "No prediction available"
```

Example usage

```
print("Next Word:", predict_next_word('the', 'stock'))
```

Morphological Analysis in NLP: Stemming and Lemmatization with NLTK

Step 1: Install NLTK Library

- First, install the NLTK (Natural Language Toolkit) library, which provides tools for working with human language data.

```
!pip install nltk
```

- Step 2: Import Required Libraries
- Import the necessary libraries from NLTK for stemming, lemmatization, tokenization, and working with wordnet.

```
import nltk  
from nltk.stem import PorterStemmer, WordNetLemmatizer  
from nltk.corpus import wordnet  
from nltk.tokenize import word_tokenize
```

Morphological Analysis in NLP: Stemming and Lemmatization with NLTK

Step 3: Download NLTK Resources

- Download the necessary NLTK resources such as tokenizers, wordnet, and POS taggers.

```
nltk.download('punkt')  
nltk.download('wordnet')  
nltk.download('averaged_perceptron_tagger')
```

Step 4: Define the Stemming Function

- Define a function to perform stemming on a list of words using the PorterStemmer.

```
def stem_words(words):  
    ps = PorterStemmer()  
    return [ps.stem(word) for word in words]
```

Morphological Analysis in NLP: Stemming and Lemmatization with NLTK

Step 5: Define a Function to Map POS Tags for Lemmatization

Define a function to map part-of-speech tags to the format required by the lemmatizer.

```
def get_wordnet_pos(word):  
    """Map POS tag to first character lemmatize() accepts"""  
    tag = nltk.pos_tag([word])[0][1][0].upper()  
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN, "V": wordnet.VERB,  
                "R": wordnet.ADV}  
  
    return tag_dict.get(tag, wordnet.NOUN)
```

Step 6: Define the Lemmatization Function

- Define a function to perform lemmatization on a list of words using the WordNetLemmatizer and the POS tags.

```
def lemmatize_words(words):  
    lemmatizer = WordNetLemmatizer()  
    return [lemmatizer.lemmatize(word, get_wordnet_pos(word)) for word in  
            words]
```

Morphological Analysis in NLP: Stemming and Lemmatization with NLTK

Step 7: Define a Function to Preprocess Text

Define a function to preprocess text by converting it to lowercase and tokenizing it into words.

```
def preprocess_text(text):  
    return word_tokenize(text.lower())
```

Step 8: Apply the Preprocessing, Stemming, and Lemmatization

- Use the defined functions to preprocess a sample text, perform stemming, and perform lemmatization. Print the results.
- `text = "Graph-based text mining involves representing text data as a graph and using graph algorithms to extract meaningful patterns."`
`words = preprocess_text(text)`

```
stemmed_words = stem_words(words)  
lemmatized_words = lemmatize_words(words)
```

```
print("Original Words:", words)  
print("Stemmed Words:", stemmed_words)  
print("Lemmatized Words:", lemmatized_words)
```